Project Specification: Search and Lost-Demand Analysis for an Odoo Online Shop

Oct 25, 2026 · @khalil, Akram

1. Overview

We will build an Odoo online shop whose search runs on Elasticsearch, and which turns every search and click into restocking decisions inside Odoo.

Item	Detail
Project type	Final-year project: ERP + Big Data
ERP side	Odoo 19 Community: eCommerce, Sales, Inventory, Purchase
Big Data side	Elasticsearch and Kibana: search engine, event store, live dashboards
Data	Amazon ESCI (real shopper queries), REES46 (real shop clickstream), simulated shoppers
Budget	Zero: free tools, one student laptop
Deliverables	Working shop and Odoo module, dashboards, evaluation report

2. Why this project

Small online shops lose sales in two ways that their sales reports never show:

- **Customers don't find what the shop has.** A typo, a different word ("sneakers" vs "trainers") or poor ranking returns nothing useful, and the customer leaves.
- **Customers want what the shop doesn't have.** Sales data only records what was sold, so nobody sees the searches that ended with no result. Even Odoo 20's new reorder suggestions work from past demand recorded in Odoo, and failed searches are not part of it.

Why we chose it:

- **Real users everywhere.** Any small business selling online faces this, and Odoo Community lets them run a shop at no license cost.
- **Each technology does what it is best at.** Odoo runs the business (products, stock, sales, purchases). Elasticsearch handles search and large volumes of events.
- **Measurable.** Public datasets with real shopper queries let us prove the results with numbers, not only with a demo.
- **Free and feasible.** Every tool is free, and everything runs on one laptop.
- **Skills used in industry.** Search engines, event analytics and ERP development are all in demand.

3. Real-life use and impact

Customers find more of what the shop sells, and the owner stocks what customers actually ask for.

Who	Today	With our project
Shopper	A typo or a different word returns nothing; out-of-stock items show first	Finds products despite typos and synonyms; in-stock items first; suggestions while typing
Shop owner	Sees sales only	Sees what customers searched for, found and did not find, live
Buyer	Restocks from past sales and guesswork	Gets restock proposals and a "customers asked for" list inside Odoo

How it helps:

- **Fewer lost sales.** Every failed search becomes visible and fixable: add a synonym, restock, or add the product.
- **Better purchasing.** Money goes to products people are asking for, not only to what sold last month.
- **No license cost.** It runs on Odoo Community as one add-on module, so a small shop can adopt it.
- **Reusable.** The same module fits any Odoo shop: clothing, electronics, pharmacy, spare parts.

4. Objectives

By the end of the project, the system will:

1. **Search.** Replace the shop's built-in search with Elasticsearch: relevance ranking, typo tolerance, synonyms, autocomplete, filters, and in-stock items first.
2. **Capture.** Record every search, click, add-to-cart and purchase as an event in Elasticsearch, visible in Kibana within seconds.
3. **Detect.** Find lost demand: searches with no results, and products in high demand that are out of stock.
4. **Act.** Turn what it finds into Odoo actions: restock proposals, draft purchase orders, and a "customers asked for" list for the buyer.
5. **Prove.** Measure search quality, detection accuracy and speed against the targets in section 11.

5. Users and scope

User	Works in	Main need
Shopper	Shop website	Find products fast, even with typos
Shop manager	Kibana dashboards, Odoo reports	See what customers search for and do
Buyer	Odoo Purchase and Inventory	Know what to restock or add
Administrator	Odoo settings	Manage synonyms, indexing and the module

In scope

- Shop search powered by Elasticsearch, delivered as a custom Odoo module
- Event capture, Kibana dashboards, the lost-demand job and purchase proposals
- Simulated shoppers, REES46 replay and the evaluation

Out of scope

- Real payments and real customer data
- Product recommendations ("customers also bought")
- Languages other than English (Elasticsearch supports them; left for later)

- Multi-server or cloud deployment, and Kafka, Spark or Hadoop: not needed at this volume on one machine

6. Functional requirements

Thirteen requirements; the nine marked Must make up the core demo.

ID	Requirement	Priority
FR1	Index Odoo products (name, description, category, brand, price, stock) in Elasticsearch; a change shows in search within 1 minute	Must
FR2	Rank search results by relevance, tolerate typos and understand synonyms	Must
FR3	Rank in-stock products above out-of-stock ones	Must
FR4	Suggest searches and products while the shopper types	Should
FR5	Filter results by category, brand, price range and stock	Should
FR6	Log every search (query, number of results), click, add-to-cart and purchase to Elasticsearch	Must
FR7	Kibana dashboards: live activity, purchase funnel, top searches, zero-result searches, demand by category	Must
FR8	A scheduled job finds lost demand: repeated zero-result searches, and high-demand products with no stock	Must
FR9	Lost demand creates restock proposals or draft purchase orders in Odoo, approved by the buyer	Must
FR10	A "customers asked for" list in Odoo, with search counts and trend; the buyer marks each item as handled	Should
FR11	Virtual shoppers replay real ESCI queries against the shop	Must

ID	Requirement	Priority
FR12	REES46 events stream into Elasticsearch at an adjustable speed	Must
FR13	Semantic search with embeddings, as an option beside keyword search	Could

7. Non-functional requirements

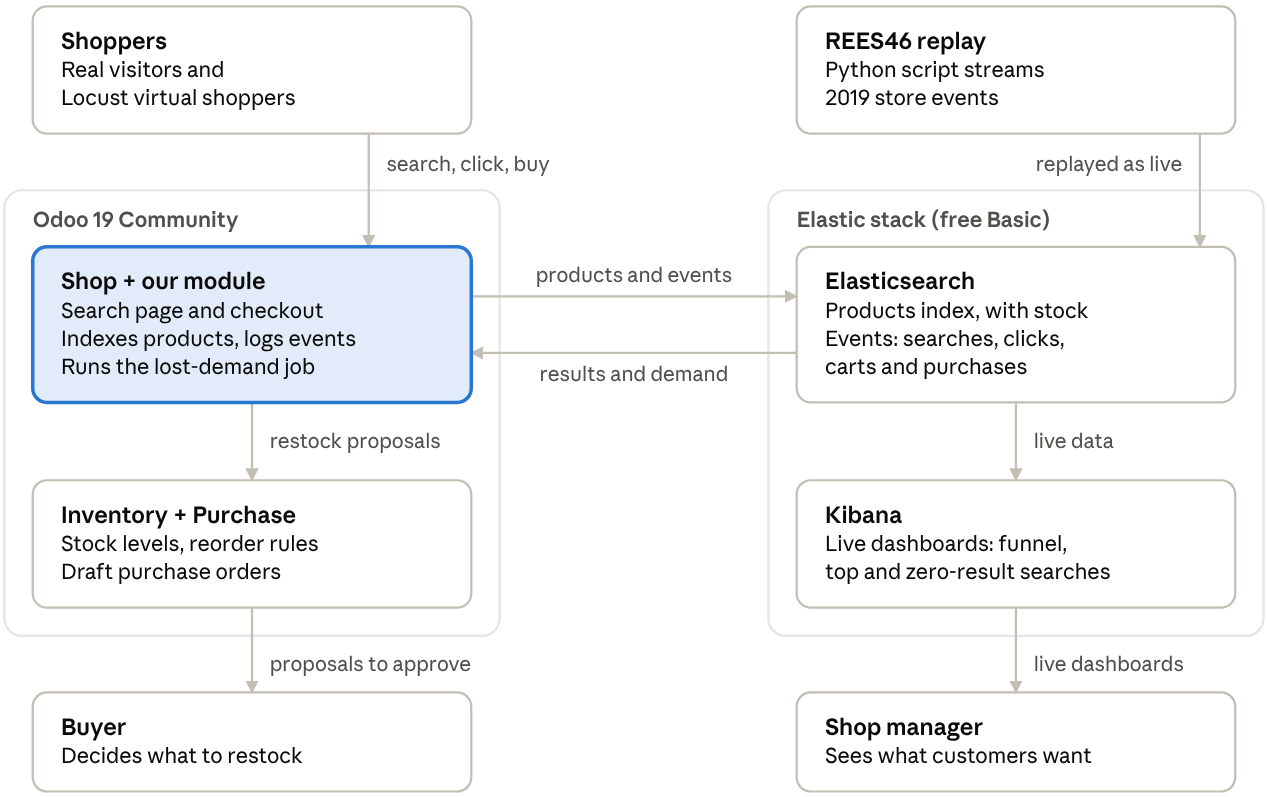
The system must run well on one student laptop, with free tools only.

Area	Requirement
Speed	The search page loads in under 1 second for 95% of requests with 50 virtual shoppers; the Elasticsearch query itself takes under 100 ms
Freshness	An event shows in Kibana less than 10 seconds after it happens
Volume	Holds at least 10 million events on one laptop
Hardware	16 GB RAM recommended; 8 GB works with reduced memory settings
Cost	Free tools only; Elasticsearch stays on its free Basic license
Reliability	If Elasticsearch is down, the shop falls back to Odoo's own search
Security and privacy	Only the Odoo server talks to Elasticsearch, with a password; events carry anonymous session IDs, no personal data
Setup	The whole stack starts with one Docker Compose command; code and setup guide live on GitHub

8. How it works

Our module links Odoo and Elasticsearch in both directions

Highlighted: our custom Odoo module. Odoo keeps its data in PostgreSQL.

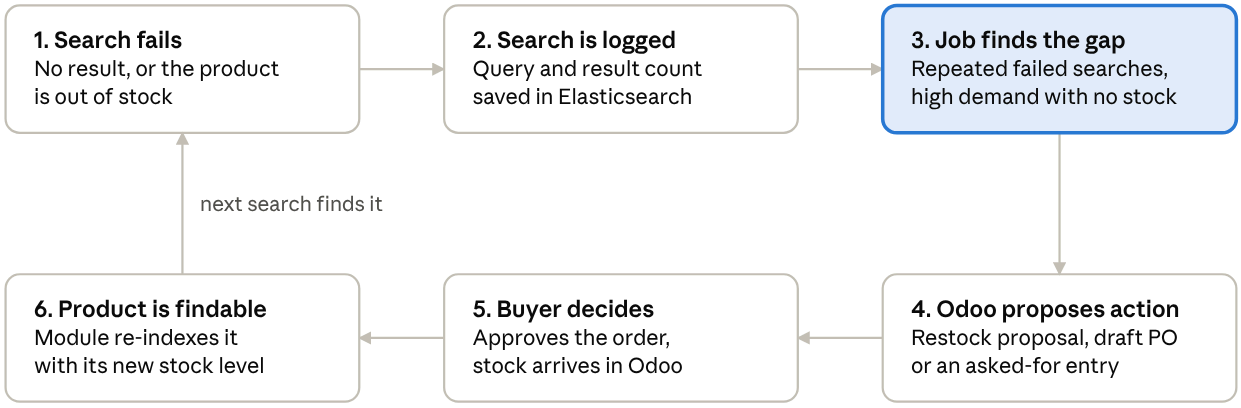


architecture · 8 parts, 8 flows

Shoppers search through our module, and Elasticsearch answers and records every event. The lost-demand job turns those events into restock proposals that the buyer approves in Odoo.

A failed search becomes a restock decision, then a sale

Highlighted: the new step this project adds. Steps 1 to 3 use Elasticsearch data; 4 to 6 happen in Odoo.



lost-demand loop · 6 steps

The loop then starts again with the next search, so each failed search can end as stock on the shelf.

9. Data

Two real public datasets and one simulator give us real queries, real shopping behavior and live traffic.

Source	What it gives us	Size	How we use it
Amazon ESCI (paper)	Real shopper queries; products with title, description, brand and color; human relevance labels (Exact, Substitute, Complement, Irrelevant)	About 130,000 queries and 2.6 million labeled query-product pairs, in English, Japanese and Spanish	English products become the shop catalog in Odoo; queries drive the virtual shoppers; labels score search quality

Source	What it gives us	Size	How we use it
REES46 clickstream	Events from a large online store: view, add to cart, remove from cart, purchase, with time, product, category, brand, price and session	About 110 million events for October and November 2019	Replayed into Elasticsearch to test dashboards and volume
Virtual shoppers (Locust)	Live traffic on our own shop: searches, clicks, add-to-cart, purchases	As much as we need	Runs the whole loop end to end and load-tests search speed

Is REES46 real-time? No, it is recorded history from 2019. But every event has a timestamp, so a small Python script replays the events in order, shifted to the present, at a chosen speed (for example, one hour of 2019 traffic per minute). Kibana dashboards on auto-refresh then show it like live traffic, the way a real shop's events would arrive. Both event sources use the same format, so the same dashboards work for both.

10. Tools and technologies

Every tool is free, and each one has a clear job; nothing is there just to add a logo.

Tool	Role in the project	Why it is worth learning
Odoo 19 Community	ERP: shop, products, stock, sales, purchases; hosts our custom module	Widely used open-source ERP. We pick 19 over the new Odoo 20 (Sept 2026) for its year of fixes and tutorials
PostgreSQL	Odoo's database	Ships with Odoo; the most common open-source database
Elasticsearch	Search engine and event store	Industry standard for search and event analytics; the free Basic license includes vector search

Tool	Role in the project	Why it is worth learning
Kibana	Live dashboards over the events	Made for Elasticsearch; charts without code
Python 3 (Elasticsearch client, pandas)	Odoo module, data import, replay script, evaluation	Odoo is written in Python, so one language covers the whole project
PyCharm	IDE for the Odoo module and scripts; runs and debugs Odoo with breakpoints	Odoo is Python, so it fits the ERP side. Core features are free and students get PyCharm Pro free
Locust	Virtual shoppers and load tests	Tests are plain Python, with live charts of response times; MIT license
Docker and Docker Compose	Run PostgreSQL, Elasticsearch, Kibana and Odoo with one command	Standard way to ship software; Docker Desktop is free for education
Git and GitHub	Version control, task board, code sharing	Expected in every software job
sentence-transformers (optional)	Embeddings for semantic search (FR13)	Runs on a laptop CPU; no paid service needed

Deliberately left out: Kafka, Spark and Hadoop. At this volume on one machine, Elasticsearch handles the load alone, and those tools would add setup work without adding results. We also avoid Elastic's paid features, such as machine-learning anomaly detection and its built-in ELSER/E5 models; any machine learning runs in Python.

11. Evaluation and success criteria

Success is measured with numbers, not only shown in a demo.

What we measure	How	Target
Search quality	NDCG@10 (a 0 to 1 score of how well the top 10 results are ordered) on English ESCI test queries: Odoo's default search vs plain Elasticsearch vs our tuned search	Tuned search scores highest of the three
Lost-demand detection	Hide 20% of the catalog from the shop, keep shoppers searching for it, and check whether the report finds it	At least 80% of hidden products flagged
Search speed	Locust with 50 virtual shoppers	Page under 1 s for 95% of requests; query under 100 ms
Freshness	Delay from event to Kibana	Under 10 s
Volume	REES46 replay	10 million events or more; dashboards load in under 5 s
Usefulness	Demo to at least one real shop owner, with a short interview before and after	Their feedback included in the report

The hidden-catalog test gives us a known answer in advance: we know exactly which products were removed, so we can check whether the system rediscovers them.

12. Order of work and deliverables

Steps run in this order, each building on the one before. Must steps come first so a working core exists early; Should and Could steps happen only if time allows.

Order	Step	What it covers	Priority
1	Setup	Docker Compose stack (Odoo 19, PostgreSQL, Elasticsearch on the Basic license, Kibana), GitHub repository, Odoo debugging in PyCharm; first interview with a local shop	Must

Order	Step	What it covers	Priority
2	Catalog import	A slice of the ESCI catalog in Odoo, as products with stock levels	Must
3	Search engine	FR1 to FR3, plus the fallback to Odoo's own search. Checkpoint: search works	Must
4	Events and dashboards	FR6 and FR7: every event logged, Kibana dashboards live	Must
5	Virtual shoppers	FR11: Locust replays ESCI queries against the shop	Must
6	Lost-demand loop	FR8 and FR9: the scheduled job and restock proposals in Odoo. Checkpoint: the full loop works	Must
7	REES46 replay	FR12: 2019 events streamed at an adjustable speed	Must
8	Evaluation	The tests in section 11	Must
9	Shop extras	FR4, FR5 and FR10: autocomplete, filters, "customers asked for" list	Should
10	Semantic search	FR13: embeddings beside keyword search	Could
11	Pilot, report, presentation	Demo to the shop owner, final report, slides	Must

Deliverables

- Source code on GitHub: the Odoo module, import scripts, replay script, Locust shoppers and the Docker Compose file
- A working demo: the shop, the Kibana dashboards and the Odoo screens
- Evaluation results: search quality, detection, speed and volume
- Setup guide, final report and presentation

13. Risks

Risk	Effect	How we handle it
Laptop RAM is too small	Slow or crashing services	Cap Elasticsearch memory at 1 to 2 GB, start Kibana only when needed, work on data slices
Importing many products into Odoo is slow	Weeks lost on data loading	Import a slice (about 20,000 products) with a batch script; Elasticsearch can hold more for tests
Elastic's start-local script starts a one-month trial of paid features	We build on a feature that disappears	Switch Elasticsearch to the free Basic license on day one
Odoo's website code changes between versions	Rework if we upgrade mid-project	Stay on Odoo 19 all year; keep all our changes in one module
REES46 is large (about 110 million events in two files)	Disk and memory pressure	Stream the files in chunks; replay a week or a month, not everything
No real shop to test with	Weaker real-life evidence	Contact local shops early; ESCI queries are already real shopper searches

Sources

- Shopping Queries Dataset (ESCI), arXiv
- Study using the REES46 e-commerce events, Atlantis Press
- What's new in Odoo 20, ECOSIRE
- Unified PyCharm overview, JetBrains
- Free PyCharm for students, JetBrains
- Locust, GitHub
- Docker Desktop license agreement, Docker Docs
- Elastic start-local, GitHub
- Elastic Cloud Hosted pricing and feature tiers